EMBER

Worth the craving.

Restaurant Ordering Website

CSCI426 - Advanced Web Programming
Project Phase 1 - Project Report

Student: Jaafar Sabra
Student ID: 72230515
Instructor: Ghadir Khalil
University: Lebanese International University (LIU)
Project mode: Individual
Updated: 13 November 2026

GitHub: github.com/JaafarSabra/EMBER-Restaurant
Website: ember-restaurant-rust.vercel.app

1. Abstract

EMBER is a responsive restaurant ordering frontend built with React and plain JavaScript. Customers browse and search a varied menu, customize products, manage their bag, choose pickup or delivery, and view simulated order tracking. The website includes Home, Our Story, Experience, Menu, Contact, Checkout and Tracking.

All React components use .js files and React.createElement calls. Browser localStorage preserves the bag, latest order and contact message. No real order, payment or email is transmitted. The interface combines warm ivory, charcoal, orange accents and local food imagery.

2. System design

src/main.js mounts the application. src/App.js owns route selection, shared cart state, customization and order creation. Hash navigation selects one of seven page components without a full page reload.

src/components/UI.js contains reusable controls, product cards, dialogs, image viewing, the map and motion behavior. src/data/menu.js holds the catalog and formatting helpers; src/data/customization.js defines choices and prices. style.css provides responsive layouts and transitions. public/assets contains the local photographs.

Ordering flow: Menu → Customize → Bag → Checkout → Tracking. Unit price equals the product base price plus paid selections and extras. Quantity multiplies the unit price. Delivery adds $3; pickup has no fee. The cart key distinguishes different selections and notes.

Checkout uses native form constraints, saves an order snapshot and clears the bag. localStorage keys ember-cart, ember-order and ember-contact persist data in the current browser. Tracking stages are simulated from the saved timestamp.

3. Technologies used

React 19: components, hooks and state.
JavaScript ES modules: all components are plain .js files using React.createElement; no JSX syntax.
HTML5 and CSS: page structure, responsive design and motion.
esbuild: bundles the JavaScript modules, React and CSS. Vite is not used.
Node.js and npm: development commands and dependency management.
localStorage: browser-local persistence.
Git and GitHub: source control and repository.
Vercel: static frontend hosting using the Other preset, npm run build and dist output.
Google Maps and Google Fonts: map embedding and typography.

The build script copies public assets and index.html to dist, bundles src/main.js into app.js, and emits app.css. npm run dev starts a local development server. No environment variables are required.

Assets and assistance: images combine OpenAI-generated imagery and Unsplash photographs. The flame wordmark and vector icons are authored in source. AI assistance was used for design, implementation and documentation. No pre-built website template was used.

4. Code snippets - key parts only

Menu search and filtering - src/pages/Menu.js

The category and text search constraints are applied together.

```
const cats = ["All", ...new Set(menu.map((x) => x.category))],
    filtered = menu.filter(
      (x) =>
        (cat === "All" || x.category === cat) &&
        (x.name + " " + x.description)
          .toLowerCase()
          .includes(search.trim().toLowerCase()),
    );
```

Plain JavaScript React rendering - src/main.js

```
import React from "react";
import { createRoot } from "react-dom/client";
import App from "./App.js";
import "../style.css";
createRoot(document.getElementById("root")).render(
  React.createElement(App, null),
);
```

4. Code snippets - continued

Customization pricing - src/App.js

Paid options and extras contribute to the unit price.

```
  const extraPrice = config
    ? config.groups.reduce(
        (sum, g) =>
          sum +
          (g.options.find((o) => o.label === choices[g.name])?.price || 0),
        0,
      ) +
      config.extras
        .filter((x) => extras.includes(x.label))
        .reduce((sum, x) => sum + x.price, 0)
    : 0;
  const selectionSummary = config
    ? [...config.groups.map((g) => choices[g.name]), ...extras, ...removals]
        .filter(Boolean)
        .join(" \xB7 ")
    : "";
```

Cart quantity updates - src/pages/Checkout.js

```
const change = (key, n) =>
    setCart((c) =>
      c
        .map((x) =>
          x.key === key ? { ...x, qty: Math.min(99, x.qty + n) } : x,
        )
        .filter((x) => x.qty > 0),
    );
```

4. Code snippets - continued

Cart persistence - src/App.js

The effect saves the bag after every cart update.

```
useEffect(() => {
    localStorage.setItem("ember-cart", JSON.stringify(cart));
  }, [cart]);
```

Checkout validation and order creation

Checkout.js renders required name and phone inputs with React.createElement. A phone pattern constrains the permitted characters; delivery also requires an address. The valid form passes FormData values to placeOrder in App.js. This function saves totals and items, clears the bag and navigates to Tracking.

```
const placeOrder = (data) => {
    const subtotal = cart.reduce((a, x) => a + x.qty * x.unitPrice, 0),
      fee = mode === "Delivery" ? 3 : 0,
      newOrder = {
        ...data,
        id: "EM" + String(Date.now()).slice(-6),
        time: Date.now(),
        mode,
        subtotal,
        fee,
        total: subtotal + fee,
        items: cart,
      };
    localStorage.setItem("ember-order", JSON.stringify(newOrder));
    setOrder(newOrder);
    setCart([]);
    location.hash = "tracking";
  };
```